Boolean Expression Evaluation

This is one of those problems where **the implementation is much easier than the intuition**.

The biggest mistake people make is trying to parse the expression like a compiler. **You don't need to.**

Instead, think about **how humans solve brackets.**

Step 1: Observe the Expression

Suppose we have

```
!(&(f,t))
```

Look carefully.

```
!(
    &(
        f,
        t
     )
 )
```

Which operation should be performed first?

Obviously

```
&(f,t)
```

because it is the innermost bracket.

After evaluating,

```
&(f,t)
= f
```

expression becomes

```
!(f)
```

Then

```
!(f)
= t
```

Finished.

This is exactly how arithmetic expressions work.

```
2*(3+4)

First
3+4

Then

2*7
```

So the problem is basically

> **Evaluate the deepest expression first.**

Whenever you hear this,

> **Think Stack.**

Step 2: Why Stack?

Suppose

```
|(&(t,f),!(f),t)
```

Let's read from left to right.

```
|
(
&
(
t
```

```
,
f
)
,
!
(
f
)
,
t
)
```

Notice something.

The last '(' opened is always closed first.

That is

```
(
    (
        )
)
```

This is exactly

Last Opened

↓

First Closed

which is

```
LIFO
```

which is exactly what a stack does.

So stack is the natural data structure.

Step 3: What should we push?

Read every character.

Ignore

```
(
,
```

because they don't affect computation.

Push

```
&
|
!
t
f
```

onto stack.

Example

Expression

```
!(&(f,t))
```

Reading

```
!
```

Stack

```
!
```

Read '('

ignore

Read '&'

Stack

```
!
&
```

Read '('

ignore

Read

```
f
```

Stack

```
!
&
f
```

Read ','

ignore

Read

```
t
```

Stack

```
!
&
f
t
```

Now we see

```
)
```

Aha!

A complete expression has ended.

Now we can evaluate.

Step 4: What happens when ')' comes?

This is the whole trick.

Suppose stack contains

```
!
&
f
t
```

Top

```
t
```

Below

```
f
```

Below

```
&
```

How do we know where operands stop?

Simple.

Keep popping until operator appears.

Pop

```
t
```

Pop

```
f
```

Next

```
&
```

Operator found.

Now we know

```
&(f,t)
```

Step 5: Evaluate

For AND

Need to know

```
Any false?
```

If yes

```
false
```

Otherwise

```
true
```

For OR

Need to know

```
Any true?
```

If yes

```
true
```

Else

```
false
```

For NOT

Just flip

```
t → f
f → t
```

After computing,

Push result back.

Stack

```
!
f
```

Continue reading.

Eventually

```
)
```

again.

Pop

```
f
```

Operator

```
!
```

Evaluate

```
!(f)=t
```

Push

```
t
```

End.

Return

```
true
```

Complete Intuition

Imagine every ')' says

> "Everything inside my brackets is complete. Compute it now."

The stack already contains everything needed.

So every time ')' comes,

1. Collect operands.
2. Find operator.
3. Compute.
4. Push answer back.

This gradually reduces

```
&(t,|(f,t),!(f))
```

into

```
&(t,t,t)
```

then

```
t
```

Dry Run

Let's dry run

```
|(f,f,f,t)
```

Initially

```
stack = []
```

Read

```
|
```

```
|
```

Read '('

ignore

Read

```
f
```

Stack

```
|
f
```

Read comma

ignore

Read

```
f
```

Stack

```
|
f
f
```

Read comma

ignore

Read

```
f
```

Stack

```
|
f
f
f
```

Read comma

ignore

Read

```
t
```

Stack

```
|
f
f
f
t
```

Now

```
)
```

Collect operands.

Pop

```
t
```

Pop

```
f
```

Pop

```
f
```

Pop

```
f
```

Now operator

```
|
```

Operands are

```
f
f
f
t
```

OR

```
false OR false OR false OR true

=
true
```

Push

```
t
```

Stack

```
t
```

Done.

Answer

```
true
```

Another Dry Run

Expression

```
&(|(f),!(f),t)
```

Read until first ')'

Stack

```
&
|
f
```

Encounter ')'

Evaluate

```
|(f)=f
```

Stack

```
&
f
```

Continue

Read

```
!
f
```

Stack

```
&
f
!
f
```

Encounter ')'

Evaluate

```
!(f)=t
```

Stack

```
&
f
t
```

Read

```
t
```

Stack

```
&
f
t
t
```

Encounter final ')'

Operands

```
f
t
t
```

Operator

```
&
```

AND

Since one operand is false

```
false
```

Push

```
f
```

Finished.

Answer

```
false
```

The Algorithm

```
for every character

    if '(' or ','
        ignore

    else if ')' encountered

        pop all operands

        pop operator

        evaluate

        push result

    else
        push current character
```

Why does this always work?

The key invariant is:

> **Before every `)`, the stack contains exactly the operator for that subexpression followed by all of its already-evaluated operands.**

For example, while processing:

```
&(!(f),|(t,f))
```

When the first `)` is reached, the stack looks like:

```
&
!
f
```

The `!` subexpression is evaluated and replaced by `t`:

```
&
t
```

Later, the `|` subexpression is evaluated similarly, so just before the final `)` the stack becomes:

```
&
t
t
```

At that moment, the outer `&` has all of its operands already reduced to a single `t` or `f`, so evaluating it is straightforward.

This is why **nested expressions never cause problems**—every inner expression is reduced to a single boolean before its parent is evaluated. That "inside-out" evaluation order is exactly what the stack naturally enforces.

Here is the clean stack-based C++ solution. I'll explain every part so you understand **why** it works.

```
C++
class Solution {
public:
    bool parseBoolExpr(string expression) {

        stack<char> st;

        for (char ch : expression) {

            // Ignore commas and opening brackets
            if (ch == ',' || ch == '(')
                continue;

            // If current character is not ')', push it
            if (ch != ')') {
                st.push(ch);
            }
            else {

                int t = 0;
```

```
                int f = 0;

                // Collect all operands
                while (st.top() == 't' || st.top() == 'f') {
                    if (st.top() == 't')
                        t++;
                    else
                        f++;

                    st.pop();
                }

                // Operator
                char op = st.top();
                st.pop();

                // Evaluate
                if (op == '!') {
                    st.push(f == 1 ? 't' : 'f');
                }
                else if (op == '&') {
                    st.push(f > 0 ? 'f' : 't');
                }
                else { // '|'
                    st.push(t > 0 ? 't' : 'f');
                }
            }
        }

        return st.top() == 't';
    }
};
```

Let's understand the code line by line

Step 1

```
C++
stack<char> st;
```

The stack stores

- operators (`!`, `&`, `|`)
- operands (`t`, `f`)

Example

For

```
!( &(f,t) )
```

the stack may look like

```
Top
t
f
&
!
Bottom
```

Step 2

```
C++
for(char ch : expression)
```

Read every character one by one.

Suppose

```
|(f,f,t)
```

We'll process

```
|
(
f
,
f
,
t
)
```

Step 3

Ignore useless characters

```
C++
if(ch==',' || ch=='(')
    continue;
```

Why?

Because

```
&(t,f)
```

The comma doesn't help evaluate.

The '(' doesn't help either.

Only these matter

```
&
t
f
```

Step 4

Push everything except ')'

```
C++
if(ch != ')')
    st.push(ch);
```

For

```
&(t,f)
```

Stack becomes

```
&
t
f
```

Step 5

Now suppose we encounter

```
)
```

This means

> The current subexpression is complete.

Now evaluate it.

Step 6

Count operands

```
C++
int t = 0;
int f = 0;
```

Instead of storing all operands,

we only count

```
true count
false count
```

Because

For AND

```
Need to know

Any false?
```

For OR

```
Need to know

Any true?
```

We don't need the exact list.

Step 7

Pop operands

```
C++
while(st.top()=='t' || st.top()=='f')
```

Suppose stack

```
&
t
f
t
```

Top

```
t
```

Pop

```
t = 1
```

Next

```
f
```

Pop

```
f = 1
```

Next

```
t
```

Pop

```
t = 2
```

Now top is

```
&
```

Stop.

Step 8

Get operator

```
C++
char op = st.top();
st.pop();
```

Now

```
op='&'
```

Step 9

Evaluate

NOT

```
C++
if(op=='!')
```

There is exactly one operand.

Possible cases

```
!(t)
```

counts

```
t=1
f=0
```

Need

```
f
```

So

```
C++
st.push('f');
```

Second case

```
!(f)
```

counts

```
t=0
f=1
```

Need

```
t
```

So

```
C++
st.push(f==1 ? 't':'f');
```

AND

```
C++
else if(op=='&')
```

Need

```
Any false?
```

If yes

```
false
```

Otherwise

```
true
```

So

```
C++
if(f>0)
    push 'f'
else
    push 't'
```

OR

Need

```
Any true?
```

If yes

```
true
```

Else

```
false
```

Exactly

```
C++
if(t>0)
    push 't'
else
    push 'f'
```

Step 10

Finally

Only one character remains.

```
C++
return st.top()=='t';
```

Complete Dry Run

Expression

```
!(&(f,t))
```

Read

```
!
```

Stack

```
!
```

Read

```
&
```

Stack

```
!
&
```

Read

```
f
```

Stack

```
!
&
f
```

Read

```
t
```

Stack

```
!
&
f
t
```

Read

```
)
```

Count

```
t=1
f=1
```

Operator

```
&
```

Since

```
f>0
```

Result

```
f
```

Push

Stack

```
!
f
```

Read last

```
)
```

Count

```
t=0
f=1
```

Operator

```
!
```

Result

```
t
```

Push

```
t
```

Return

```
true
```

Time Complexity

- Every character is pushed **at most once** and popped **at most once**.
- Therefore, the total work is **O(n)**, where `n` is the length of the expression.

Space Complexity

- In the worst case (deeply nested expressions), the stack can store **O(n)** characters.

This is the optimal solution for this problem.